Indian Institute of Technology Delhi – Abu Dhabi

ACOL 216: Introduction to Computer Architecture
Semester II, 2025–26

RISC-Tac-Toe

Technical Report – Assignment 1

Name: *Mohammad Taqi Ahsan*

Entry Number: *24A1CSEB0064*

Course Code: *ACOL216*

1 Overview

This report describes the design and implementation of a 6×6 Tic-Tac-Toe state verifier written in SimpleRISC assembly. The program reads a board snapshot from memory and determines whether the state is legal or illegal, and if legal, whether Player 1 wins, Player 2 wins, or the game is a draw.

The program produces exactly one output value written to a designated memory address:

- `-1`: illegal state
- `0`: legal, draw
- `1`: Player 1 wins
- `2`: Player 2 wins

2 Memory Layout

2.1 Input Board

The board is stored in row-major order starting at base address `1000`. Each cell occupies one 4-byte word. Cell (i, j) (row i, column j, both zero-indexed) is located at:

$$address(i, j) = 1000 + 4 \times (6i + j)$$

This gives a total footprint of $36 \times 4 = 144$ bytes, occupying addresses `1000` through `1143`.

2.2 Output

The result is written to address `1144` (i.e., `base + 144`), immediately following the board in memory.

2.3 Register Usage

`Register`	**Role**
`r0`	Board base address (constant throughout execution)
`r1`	Player 1 marker count / Player 1 win flag
`r2`	Player 2 marker count / Player 2 win flag
`r3`	Current cell value (scratch)
`r4`	Computed cell address (scratch)
`r5`	Cell value under inspection
`r6`	Difference `count1 - count2` (legality check)
`r7`	Outer loop index (row)
`r8`	Inner loop index (column)
`r9, r10`	Neighbor cell values (scratch)
`r13`	Status register (unused after refactor)

3 Program Design

The program is structured into five sequential phases.

3.1 Phase 1 – Cell Validation and Marker Counting

The program iterates over all 36 cells in flat order. For each cell it loads the value and checks it against $\{0, 1, 2\}$. Any other value immediately triggers the illegal branch. Valid cells contribute to either `count1` (register `r1`) or `count2` (register `r2`).

3.2 Phase 2 – Legality Check

After counting, two conditions are verified:

1. `count2 > count1` ⇒ illegal (Player 2 cannot have moved more than Player 1)
2. `count1 - count2 > 1` ⇒ illegal (Player 1 cannot have moved more than once ahead)

3.3 Phase 3 – Win Detection

Win detection uses a *center-cell pattern check*. For each interior cell the program loads its two neighbors in a given direction and checks whether all three are equal to the same non-zero player value. Any run of three or more consecutive markers will contain at least one interior cell, so the approach correctly detects runs of length 3 through 6.

Four directions are checked in sequence, each with its own checker loop:

Direction	**Neighbor offsets (bytes)**	**Loop bounds**
Horizontal	-4, $+4$	rows 0–5, cols 1–4
Vertical	-24, $+24$	cols 0–5, rows 1–4
Main diagonal ($\searrow$)	-28, $+28$	rows 1–4, cols 1–4
Anti-diagonal ($\swarrow$)	-20, $+20$	rows 1–4, cols 1–4

The offset arithmetic follows from the row stride of $6 \times 4 = 24$ bytes and the column stride of 4 bytes. For example, the main diagonal neighbor one step up-left is at offset $-24 - 4 = -28$.

Loop bounds are chosen so that both neighbors always lie within the board, eliminating the need for explicit boundary checks inside the loop body.

After each directional checker, the program checks whether *both* win flags are set simultaneously. If so, the state is immediately classified as illegal.

3.4 Phase 4 – Judgement

After all four checkers complete, the win flags `r1` and `r2` are inspected:

- Both set ⇒ illegal
- Only `r1` set ⇒ output 1
- Only `r2` set ⇒ output 2
- Neither set ⇒ output 0 (draw)

4 Correctness Arguments

Legality. The counter loop visits every cell exactly once (36 iterations, decrementing `r5` from 36 to 0). The two count comparisons cover all cases of marker imbalance by checking both `count2 > count1` and `count1 - count2 > 1`.

Win detection completeness. Any winning run of length $k \geq 3$ contains at least one cell that is not at either end of the run. That cell has a matching left neighbor and a matching right neighbor (in the relevant direction), so the center-cell check fires. The loop bounds guarantee every such interior position is visited.

Bounds safety. By starting inner loops at index 1 and terminating before index 5 (for rows/cols), both the negative and positive neighbor loads always refer to valid board addresses. No out-of-bounds memory access occurs.

Both-win detection. Each directional overflow section checks whether both `r1` and `r2` equal 1 before proceeding. This ensures that a simultaneously winning board is caught regardless of which checker detects the second win.